EV Charge DZ

Shared Database API Report

Web Platform ↔ Shared MySQL Database ↔ Kotlin App

Project	EV Charge DZ
Authors	Fodhil Yacine & Chibani
Date	May 2026
Backend	Node.js / Express (TypeScript)
Database	MySQL 8.0 — `ev_charge_dz`
API Port	`3001`

This document summarises the API changes made to support the shared-database architecture between the admin web platform and the Kotlin mobile application.

Contents

CHAPTER 1

Context and Architecture

1.1 Goal

The project uses a **single shared MySQL database** (`ev_charge_dz`) accessed by two independent clients:

- **Web Platform** — React admin dashboard managed by Fodhil Yacine.
- **Kotlin Mobile App** — Android application managed by Chibani.

Both clients communicate with the same Express backend running on port `3001`. The teacher's requirement was that every path must return a **JSON response**, never HTML, so that the Kotlin app can consume the same API as the web platform.

1.2 Before the Change

All routes were mounted directly at the root:

```
app.use('/auth',     authRoutes);
app.use('/stations', stationRoutes);
app.use('/sessions', sessionRoutes);
// ... etc.
```

There was no clear contract between the web platform and the mobile app. The Kotlin app had no dedicated authentication or data endpoints.

1.3 After the Change

All routes now live under `/api/`. The web platform uses `/api/*` and the Kotlin app uses `/api/app/*`. Both return pure JSON. No HTML is ever served by the backend.

```
http://localhost:3001
  /health                    -- health check (no auth)
  /api/auth/*                -- web platform
   authentication
  /api/stations/*            -- web platform station
   management
  /api/sessions/*            -- web platform session
   listing
  /api/tickets/*             -- maintenance tickets
  /api/billing/*             -- billing records
  /api/alerts/*              -- system alerts
  /api/dashboard/*           -- KPI data
  /api/users/*               -- dashboard user management
  /api/tenants/*             -- tenant management
  /api/connectors/*          -- connector management
  /api/app/auth/*            -- Kotlin app authentication
  /api/app/stations/*        -- Kotlin app station map
  /api/app/sessions/*        -- Kotlin app session history
```

CHAPTER 2

Changes Made — Web Platform Side (Yacine)

2.1 Files Modified

2.1.1 server/src/index.ts — Route prefix

Added `/api` prefix to all ten existing route handlers and imported the three new app route files.

```
// Web platform routes
app.use('/api/auth',        authRoutes);
app.use('/api/tenants',     tenantRoutes);
app.use('/api/users',       userRoutes);
app.use('/api/stations',    stationRoutes);
app.use('/api/connectors', connectorRoutes);
app.use('/api/sessions',    sessionRoutes);
app.use('/api/tickets',     ticketRoutes);
app.use('/api/billing',     billingRoutes);
app.use('/api/alerts',      alertRoutes);
app.use('/api/dashboard',   dashboardRoutes);

// Kotlin app routes
app.use('/api/app/auth',      appAuthRoutes);
app.use('/api/app/stations', appStationsRoutes);
app.use('/api/app/sessions', appSessionsRoutes);
```

2.1.2 server/src/middleware/auth.ts — App user middleware

Added a second JWT payload interface (`AppAuthPayload`) and a new `requireAppAuth` middleware for Kotlin app users. Dashboard users and app users have separate identities in the database (`users` table vs `app_users` table).

```
export interface AppAuthPayload {
  appUserId: string;
  email:     string;
}

export function requireAppAuth(req, res, next): void {
  // Verifies Bearer token, populates req.appUser
}
```

2.1.3 `project/.env` — React base URL

Updated `VITE_API_BASE_URL` so the React dashboard continues to work without changing any path in the frontend code:

```
# Before
VITE_API_BASE_URL=http://localhost:3001

# After
VITE_API_BASE_URL=http://localhost:3001/api
```

2.2 New Files Created

2.2.1 `server/src/routes/appAuth.ts`

Handles `app_users` registration and login (completely separate from dashboard staff accounts).

Method	Path	Description
POST	/api/app/auth/register	Create new app user account
POST	/api/app/auth/login	Login, returns JWT token

2.2.2 `server/src/routes/appStations.ts`

Returns station data shaped for the Kotlin map view using the existing `vw_app_station_map` database view. **No authentication required** so the map loads before the user logs in.

Method	Path	Description
GET	/api/app/stations	All available stations (public)
GET	/api/app/stations/{id}	Single station detail (public)

2.2.3 `server/src/routes/appSessions.ts`

Returns the authenticated user's personal charge history using the existing `vw_app_session_history` database view.

Method	Path	Description
GET	/api/app/sessions	Logged-in user's session history

Supports optional query parameters: `limit` (default 50, max 200) and `offset` (default 0).

CHAPTER 3

For the Kotlin App Developer (Chibani)

3.1 Base URL

Replace `<server-ip>` with the IP address of the machine running the backend (use `localhost` or `10.0.2.2` for the Android emulator, or the actual LAN IP for a physical device).

```
val BASE_URL = "http://<server-ip>:3001/api/app"
```

Android emulator note: `10.0.2.2` maps to `localhost` on the host machine. Physical device on the same Wi-Fi: use the host machine's local IP (e.g. `192.168.x.x`).

3.2 Authentication Flow

3.2.1 Register

```
POST http://<server-ip>:3001/api/app/auth/register
Content-Type: application/json

{
  "email":     "user@example.com",
  "password":  "SecurePass123",
  "full_name": "Ahmed Benali",
  "phone":     "+213555000000"   // optional
}
```

Response 201:

```
{
  "token": "<jwt-token>",
  "user": {
    "appUserId": "uuid",
    "email":     "user@example.com",
    "full_name": "Ahmed Benali",
    "phone":     "+213555000000"
```

```
  }
}
```

3.2.2 Login

```
POST http://<server-ip>:3001/api/app/auth/login
Content-Type: application/json

{
  "email":    "user@example.com",
  "password": "SecurePass123"
}
```

Response 200:

```
{
  "token": "<jwt-token>",
  "user": {
    "appUserId": "uuid",
    "email":     "user@example.com",
    "full_name": "Ahmed Benali",
    "phone":     "+213555000000"
  }
}
```

Save the token in SharedPreferences or DataStore. Every authenticated request must include it in the header:

```
Authorization: Bearer <jwt-token>
```

3.3 Station Map

3.3.1 Get All Stations (public — no token needed)

```
GET http://<server-ip>:3001/api/app/stations
```

Response 200 — array of station objects:

```
[
  {
    "id":                    "uuid",
    "name":                  "Station Alger Centre",
    "address":               "Rue Didouche Mourad",
    "city":                  "Alger",
```

```
    "lat":                  36.7525,
    "lng":                  3.0420,
    "status":               "available",
    "available":            1,
    "max_power_kw":         50,
    "power":                "50 kW",
    "price":                "25 DZD/kWh",
    "total_connectors":     4,
    "available_connectors": 2,
    "connector_types":      "CCS, Type2"
  },
  ...
]
```

3.3.2 Get Single Station

```
GET http://<server-ip>:3001/api/app/stations/{id}
```

Returns a single station object with the same fields as above, or `404` if not found.

3.4 Session History (requires token)

```
GET http://<server-ip>:3001/api/app/sessions
Authorization: Bearer <jwt-token>
```

Optional query parameters:

Parameter	Default	Description
`limit`	50	Number of records to return (max 200)
`offset`	0	Skip this many records (for pagination)

Response 200 — array of session objects:

```
[
  {
    "session_id":      "uuid",
    "app_user_id":     "uuid",
    "station_id":      "uuid",
    "station_name":    "Station Alger Centre",
    "station_address": "Rue Didouche Mourad",
    "connector_id":    "uuid",
    "connector_type":  "CCS",
    "max_power_kw":    50,
    "payment_method":  "card",
```

```
    "start_time":       "2026-05-01T10:30:00Z",
    "end_time":         "2026-05-01T11:15:00Z",
    "duration_min":     45,
    "energy_kwh":       22.5,
    "cost_dzd":         562.5,
    "status":           "completed"
  },
  ...
]
```

3.5 Error Responses

All endpoints return errors in a consistent format:

```
{ "error": "descriptive message" }
```

HTTP Code	Meaning	When it happens
400	Bad Request	Missing required fields
401	Unauthorized	No token or invalid token
403	Forbidden	Account suspended
404	Not Found	Station ID does not exist
409	Conflict	Email already registered
500	Server Error	Unexpected backend error

3.6 Recommended Kotlin Integration Steps

1. Add **Retrofit** + **OkHttp** + **Gson** (or Moshi) to `build.gradle`.
2. Create a **Retrofit service interface** with the endpoints above.
3. Add an **OkHttp interceptor** that reads the JWT from DataStore and injects the `Authorization` header automatically on every request.
4. Create **data classes** matching the JSON fields (use `@SerializedName` for snake_case fields like `station_name`, `start_time`, etc.).
5. Map `vw_app_station_map` fields to a `ChargingStation` data class and `vw_app_session_history` fields to a `SessionRecord` data class.
6. For the map: load stations on `onMapReady` — no login required.
7. For session history: check token validity first; redirect to login if 401.

3.7 Quick Reference Card

Feature	Endpoint	Method	Auth
Register	`/api/app/auth/register`	POST	None
Login	`/api/app/auth/login`	POST	None
Station map	`/api/app/stations`	GET	None
Station detail	`/api/app/stations/{id}`	GET	None
Session history	`/api/app/sessions`	GET	Bearer token

CHAPTER 4

Summary

What was completed on the web platform side:

- ✓ All existing routes prefixed with `/api/` — web platform still works via updated `VITE_API_BASE_URL`.
- ✓ App user JWT middleware added (`requireAppAuth`).
- ✓ `appAuth.ts`: register & login for `app_users`.
- ✓ `appStations.ts`: public station map endpoint using `vw_app_station_map`.
- ✓ `appSessions.ts`: authenticated session history using `vw_app_session_history`.

What Chibani needs to do on the Kotlin side:

- Configure Retrofit with base URL `http://<server-ip>:3001/api/app`.
- Implement register / login screens calling `/auth/register` and `/auth/login`.
- Save the returned JWT token in DataStore / SharedPreferences.
- Inject the token via an OkHttp interceptor on authenticated requests.
- Map `vw_app_station_map` columns to a `ChargingStation` data class.
- Map `vw_app_session_history` columns to a `SessionRecord` data class.
- Load stations (no auth) for the map; load sessions (with auth) for the history screen.